

CINEIQ

An Open, Explainable Hybrid Movie Recommendation Engine

June 2026

Abstract

Today's streaming services entrap users in opaque recommendation systems that promote more products rather than providing insights about how particular movies are recommended. CINEIQ aims to fill this void by developing an open, explainable hybrid movie recommendation system that leverages collaborative filtering with Singular Value Decomposition (SVD), content-based filtering with TF-IDF cosine similarity, and a sentiment analysis-powered re-ranker based on IMDB reviews. The recommendation system provides explanations for every single recommendation, and users can visualize their taste through a taste dashboard. This application is hosted using FastAPI for the backend and Streamlit for the frontend and is built to handle up to 25 million movie ratings..

1. Introduction

1.1 Problem Statement

The process of content discovery through present-day online streaming services is beset by three main challenges: recommendations have no explanation behind them, thus leaving out any information as to why a particular film was recommended; the algorithms used are prone to being biased towards commercially-sponsored movies; and there is no escape from the recommendations made to users as they only serve to reinforce one's tastes.

1.2 Motivation

Recommendation engines have treated recommendations as black boxes up to now. It is impossible for a user receiving an engine's recommendations to know how the recommendation has been made, whether through similarity between genres, viewing history, or user feedback. The rationale behind CINEIQ was that recommendations needed to be transparent and unbiased.

1.3 Objectives

- Creates a recommender system that uses both collaborative filtering, content-based filtering, and sentiment analysis
- Constructs a re-ranking mechanism based on sentiment using IMDB review dataset
- Develops a taste profiler of users, displaying genre, decades, and ratings of movies preferred by users
- Generates human-readable explanations for every recommendation
- Deploys the application using FastAPI and Streamlit

2. Datasets

2.1 MovieLens 25M

The MovieLens 25M dataset from GroupLens contains 25 million ratings by 162,541 users across 62,423 movies. It provides the `userId`, `movieId`, and `rating` (0.5 to 5.0) columns used for collaborative filtering. The `movies.csv` file provides movie titles and pipe-separated genre labels.

2.2 TMDB Metadata

The TMDB 45K Movies dataset from Kaggle includes rich metadata for 45,000 movies: overview text, genres, cast, crew, keywords, vote average, and production details. Three files are used: `movies_metadata.csv`, `keywords.csv`, and `credits.csv`, merged on the TMDB movie ID.

2.3 IMDB 50K Reviews

The IMDB Dataset of 50K Movie Reviews from Kaggle contains 50,000 labelled movie reviews with binary sentiment labels (positive/negative). This dataset is used to train the sentiment classifier that provides the re-ranking signal in the hybrid engine.

2.4 Dataset Integration

The MovieLens `links.csv` file serves as the bridge between datasets, mapping MovieLens `movieId` values to TMDB `tmdbId` values. This enables the hybrid engine to combine collaborative signals from MovieLens ratings with content signals from TMDB metadata for the same movies.

3. System Architecture

The CINEIQ system consists of three layers in its architecture: ML Core, which is `recommender.py`, API server layer, `main.py` using FastAPI, and the third one being UI layer, `app.py` using Streamlit. On startup, FastAPI automatically detects missing models and executes the whole pipeline to create pickled models for the first time. Subsequently, the models are loaded from disk within 30-60 seconds. The Streamlit app makes HTTP calls to the FastAPI through four API endpoints, which are `/recommend`, `/similar`, `/taste`, and `/health`.

TMDB + MovieLens + IMDB → `recommender.py` → FastAPI → Streamlit UI

4. Methodology

4.1 Content-Based Filtering

Movie tags are created by combining various text features, including the synopsis (weighed twice), genres (weighed thrice), keywords (weighed once), the top three actors, and the director. TFIDF vectorizer is used with removal of English stop words. Cosine similarity is then calculated dynamically at query time based on these tags for comparison of the queried movie with all other movies.

4.2 Collaborative Filtering

Scipy `csr_matrix` sparse representation is used instead of traditional pivot table for users and movies that consumes several GBs of memory; sparse representation requires less than 100 MB of memory space to store ratings of 25M users. The truncated SVD with 100 latent factors is

applied to the sparse matrix representation, generating (162541 x 100) reduced user matrix. The computation of full (162541 x 162541) matrix cosine similarity takes up 197 GBs of memory, therefore, similarities between users will be computed upon request: one single row of similarity vector will be computed in a fraction of a second.

4.3 Sentiment Re-Ranker

Logistic Regression classifier model, using TF-IDF Feature Extractor (features = 10,000), is trained using IMDB 50K Reviews dataset. For TMDB data, since it does not contain movie ids, the classification model is used for TMDB movie tags data. This produces a sentiment score, ranging from 0 to 1, which is linked with MovieLens IDs through links.csv.

4.4 Hybrid Engine

The hybrid engine combines the three normalized signals using tunable weights α (collaborative), β (content-based), and γ (sentiment). To ensure that no engine becomes dominant based on their different dataset sizes, each signal is normalized between [0, 1]. The final score will be calculated as:

$$\text{final_score} = (\alpha \times \text{CF} + \beta \times \text{Content} + \gamma \times \text{Sentiment}) / (\alpha + \beta + \gamma)$$

This auto normalization based on weights guarantees that the scores will always be valid, irrespective of the user-chosen values. Movies watched by the user are excluded before recommending anything to the user.

4.5 Explainability Layer

All recommendations come along with human readable explanations based on rules. Content-based recommendation explanation mentions the anchor movie from the user's rating history, and genres which are shared among the movies, as well as the predominant genre of the movie to be recommended. Collaborative recommendation explanation simply says that people with similar taste liked this movie too. Dominant engine for each recommendation is found at blending time by comparing normalized values..

5. Results

5. Results

5.1 Movie Search

Content-based searching is appropriate when searching for franchise movie sequels and other films with common themes. For example, when searching for "Toy Story," the results include "Toy Story 2" and "Toy Story 3" with similarity scores greater than 0.48 and others that have common themes, with each showing genre tags, similarity score bars, and content explanations using the search query.

5.2 Hybrid Recommendations

The mixed model creates personalized suggestions for all users together with explanation for each one of them. Suggestions are influenced by the values of sliders alpha, beta, and gamma, and depending on how much the algorithm is influenced by either collaborative or content

filtering, the results will be varied. A user that gave high rating for the movie Star Wars Episode VI (1983) got Star Wars Episode IV (1977) as a suggestion.

5.3 Taste Dashboard

The dashboard clearly represents the ratings that have been given by the individual before. The radar chart indicates which genres the users like more. From the bar graph for decades, we can see whether the users like old movies or new ones, while the table shows the individual ratings for each genre.

6. Challenges and Solutions

Challenge	Solution
197GB user similarity matrix	On-the-fly per-user cosine similarity on 100-dimensional SVD vectors
Pivot table memory explosion with 25M ratings	scipy csr_matrix sparse representation — reduced to under 100MB
Signal scale mismatch across engines	Max normalization applied independently per engine before blending
IMDB reviews lack movie IDs	Sentiment classifier applied to TMDB movie tag text as proxy
Slow startup from recomputing models	Auto-pickle generation on first run, instant load on subsequent runs

7. Future Work

- Alpha/beta/gamma combinations and precision@K using MLflow experiments logging
- LIME explanations to understand content recommendation at feature level
- DistilBERT fine-tuned on movie reviews for better sentiment
- Real-time rating ingestion without retraining for collaborative filtering

8. Conclusion

The system CINEIQ achieves all the four deliverables mentioned in the problem statement: a hybrid recommendation engine, a sentiment-aware re-ranking mechanism, a taste dashboard, and an explainability module. The system deals with the huge size of the MovieLens 25M dataset by introducing several innovative approaches to engineering, such as sparse matrices and similarity calculation on the fly, thus providing a viable production solution that is possible without large memory requirements. The tunable mix of three signals and rule-based explainability allow CINEIQ to be called really open and understandable.

Find Similar Movies

Content-based recommendations using TF-IDF cosine similarity.

Results for "Jumanji"

#1 Table No. 21

Score: 0.219

Because you liked Jumanji (1995) — similar Thriller film

#2 The Mend

Score: 0.178

Because you liked Jumanji (1995) — similar Comedy film

#3 The Mend

Score: 0.178

Because you liked Jumanji (1995) — similar Comedy film

#4 Liar Game: Reborn

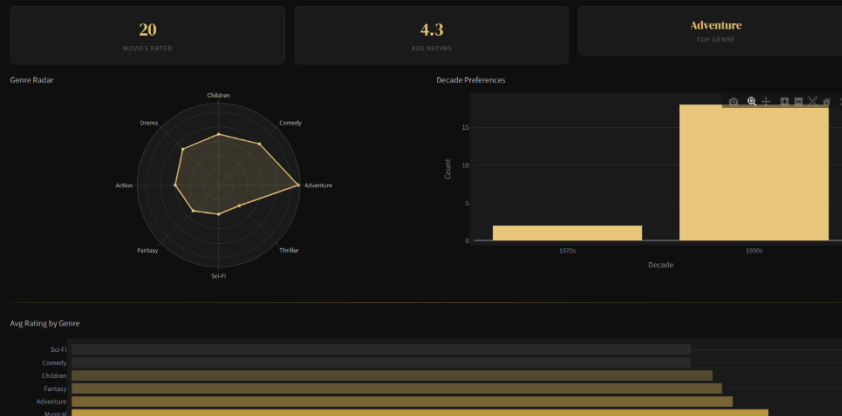
Score: 0.172

Because you liked Jumanji (1995) — similar Drama film

Your Taste Profile

Visualize your preferences from rating history.

User ID



Your Recommendations

Hybrid engine — collaborative + content + sentiment.

User ID:

Collaborative:

Content:

Sentiment:

50% Collaborative 40% Content 10% Sentiment

Top 5 picks for User 200

#1 Clocktoppers (2002) Adventure

Score: 0.441

Adventure Children Sci-Fi Thriller

Because you liked Babe (1995) — shared genres: Children

#2 Nutty Professor, The (1996) Comedy

Score: 0.398

Comedy Fantasy Romance Sci-Fi

Users with similar taste to you also rated this highly

References

- [1] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4):19:1–19:19.
- [2] TMDb 5000 Movie Dataset. Kaggle. <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>
- [3] IMDB Dataset of 50K Movie Reviews. Kaggle. <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>
- [4] Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830, 2011.
- [5] FastAPI Documentation. <https://fastapi.tiangolo.com>
- [6] Streamlit Documentation. <https://docs.streamlit.io>
- [7] Scipy Sparse Matrix Documentation. <https://docs.scipy.org/doc/scipy/reference/sparse.html>

Team Contributions

Name	Roll Number	Contribution
Ramireddy Ruthvika	250101083	ML Pipeline, Hybrid Engine, Backend
Parvathala Rishitha	250101070	Sentiment Re-Ranker, Explainability
Marpu Akshara	250101062	Streamlit Dashboard, FastAPI Integration